Save Point

Flutter Rewrite: Plan & Roadmap

QuillFin · Prepared for Logan Young, Save Point Limited · 6 November 2026

Status: **in progress, real code exists.** Originally written in the Python `Quiz-app-Ai` repo and moved here once this project stood on its own; references to `Docs/architecture.md` below that mean the *Python* app's docs are to that repo. This document is the single reference for the mobile-first Flutter rewrite — supersedes ad-hoc chat notes. The existing Python/Tkinter app (`Docs/architecture.md` etc.) is untouched and keeps working; this describes a new, separate app that reuses its *ideas* (and, where useful, its algorithms ported to Dart), not its code directly.

See §11 for exactly what's built, tested, and still blocked as of the last work session.

---

1. Vision

A mobile-first notes + spaced-repetition quiz app. You keep notes organized into subject folders, the app generates quiz questions from them using your own connected AI account, and reviews you on a fixed schedule that gets longer the better you know something. Visual identity matches savepoint.cc.

---

2. Decisions locked in during planning

- **Architecture: local-only, no backend, no real user accounts.** Everything lives on-device. "Account" means "which AI provider / API key / cloud-storage connections are configured," not a SavePoint login. Quiz generation happens when the app is opened and something is due — not while the app is closed. A local notification can remind the user to open it. This is a deliberate deferral, not a dead end: true always-on generation or cross-device sync would require a real backend later, and nothing here should make that harder to add on top afterward.
- **Scheduling sequence:** `1 → 2 → 3 → 5 → 7 → 14 → 30` **days.** First review 24h after a note is saved. A correct answer advances one step; reaching the end of the sequence loops all the way back to the 1-day start (not a partial/steady-state loop like the current desktop app). A wrong answer at any point resets straight back to the 1-day start. See §6 for the exact algorithm spec.
- **Integration feasibility, decided honestly up front:**
  - Dropbox, Google Drive, Microsoft OneNote — real public APIs, genuinely buildable.
  - Obsidian — no cloud API (it's local Markdown files); the realistic version of "Obsidian integration" is "point the app at your Obsidian vault folder / its synced copy," not a true API integration.

    - Apple Notes / iCloud — **no public API exists**; not realistically buildable without unsupported workarounds. Treated as out of scope.

- **No desktop → mobile data migration in v1.** The current app's SQLite data doesn't automatically carry over. If that's wanted, the practical path is exporting notes from the desktop app and re-importing them as files on mobile — not a direct DB migration. Flagging this now so it's a conscious choice, not a surprise later.
- **Design source of truth (savepoint.cc) not yet fully captured.** A first pass at the site (text/markup only, no CSS) confirmed a retro/pixel-art "Save Point" gaming aesthetic, consistent in spirit with the current app's dark neo-brutalist theme (`gui_theming.md`) — but exact hex codes/fonts weren't verifiable that way. Before final theming work, pull the actual brand assets/style guide or take real screenshots to check against `gui/theme.py`'s existing palette claims.

---

3. Information architecture

Four tabs:

1. **Notes** — the notebook. Subject/topic folders, full note CRUD, file import (single + bulk), usable standalone as a notes app.
2. **Quiz** — quizzes organized in the same folder structure as Notes, a **"Today's Quizzes"** screen surfacing what's due right now, each quiz traceable back to its source note(s), manual editing of AI-generated questions.
3. **Account** — AI provider + API key, cloud-storage connections (Dropbox/Drive/OneNote), nothing else (no login, per §2).
4. **Settings** — everything else (default question format, notification preferences, theme, data export, etc.)

---

4. Feature breakdown

Notes tab

- Folder tree (subjects), CRUD on folders and notes within them.
- Manual note creation/editing (this is the "usable as a notes app" req).
- Import single files: `.txt`, `.md`, `.pdf`, `.docx`, `.pptx`, `.xlsx` (or closest practical equivalent per format).
- **Bulk import.**
- **Import staging flow** (applies to both single and bulk import, and to cloud-source imports):
    1. User picks a source (device file picker, or a connected cloud account) and a destination folder in the app.
    2. The app imports preserving the **source's own folder structure** underneath that destination — it does not flatten or re-sort anything automatically.
    3. Before anything is committed, show a **preview screen**: the proposed folder/file tree as it would land in the app.
    4. User can **drag items around** in that preview to reorganize before committing.

5. Explicit **confirm** step commits it for real. Nothing is written to the real notebook before this step.

- Cloud sources: Dropbox, Google Drive, OneNote (OAuth-based). Obsidian via vault-folder pointing. Apple Notes/iCloud out of scope (§2).

Quiz tab

- Same folder structure as Notes (a quiz "lives" in its note's folder, or its own mirrored structure — data model detail in §5).
- **"Today's Quizzes"** screen: everything due right now, review loop (same short-answer/multiple-choice interaction model as the current app).
- Quiz → source note traceability: tapping a quiz shows which note(s) generated it (direct port of the current app's `question_source_notes` concept).
- Manual editing of AI-generated questions (prompt/expected answer/choices) — new capability, doesn't exist in the current app.
- Automatic generation: on app open, check which notes are due for new questions (per the scheduling rules) and generate via the connected AI provider, no manual button-press required (manual "regenerate/generate more" should probably still exist as an option, not removed).

Account tab

- AI provider selection + API key entry + test connection (direct concept port from the current Settings screen).
- Cloud storage connection management (connect/disconnect Dropbox/Drive/ OneNote).

Settings tab

- Default question format, notification preferences (remind me when quizzes are due), theme, and:
- **Full data export** — every note, every quiz, every review history entry, in a portable format (structured JSON + the original imported files where practical, zipped). This is explicitly a top-level requirement, not an afterthought — "get all my data out" should always work with no friction.

---

5. Data model changes vs. the current app

Starting from the current schema (`Docs/database_schema.md`) as a reference point, not as something being literally migrated:

- **Folders**, a new first-class concept. Both notes and quizzes need to live in a folder hierarchy. Likely modeled as a `folders` table (id, parent_folder_id nullable, name) with `notes.folder_id` replacing the current flat `subject_id` . Whether quizzes get their *own* folder placement or always inherit their source note's folder is an open question — leaning toward "inherit from source note" to avoid a second manual-organizing chore, but worth deciding deliberately rather than defaulting silently.
- **Quiz scheduling fields** change to match §2's new sequence — same shape as today (`interval_index` + `next_due_at`), just a shorter sequence array and "loop to index 0" instead of "loop to a fixed middle index."
- **Question ↔ source note(s)** — keep the existing many-to-many concept (`question_source_notes`), it already does exactly what "see where the quiz was made from" needs.

- **Import staging state** — the drag-and-drop preview needs *some* in-memory or lightly-persisted representation of "proposed tree, not yet committed" before it's written into the real folders/notes tables. Likely just in-memory app state (a pending tree the user edits, discarded if they back out) rather than a database concept — cheaper and there's no need for it to survive an app restart.
- **Manual quiz edits** — need an explicit "edited by user" marker distinct from "generated by AI," so the app (and the user) can tell the two apart later (e.g. don't silently overwrite a hand-edited question if it's ever regenerated).
- **Cloud connection tokens** — new table/secure-storage entries per provider (OAuth tokens), separate from the AI API key.

---

6. Scheduling algorithm spec (Dart port)

Direct port of the current app's approach (`core/scheduling.py`), same shape, new numbers:

```
INTERVAL_SEQUENCE_DAYS = [1, 2, 3, 5, 7, 14, 30]
MAX_INDEX = 6  // last valid index

new question:      interval_index = 0, first due 1 day after note save
                    (NOT immediately due, unlike the current desktop app)
correct answer:     if index >= MAX_INDEX: loop back to index 0
                    else: index += 1
wrong answer:       index = 0, regardless of prior index
```

This is a pure function, same as the Python original — straightforward to port and to unit test identically (a table of `(current_index, was_correct) -> new_index` cases, exactly like `tests/unit/core/test_scheduling.py` already does).

---

7. Proposed tech stack

Chosen to mirror the current project's own stated engineering values (`Docs/beginners_guide.md`) wherever Flutter has an equivalent option, rather than defaulting to "whatever's most popular":

- **Local database**: `sqflite` , hand-written repository classes per table group (same repository-pattern philosophy as today's `storage/*_repo.py` — explicit SQL, no ORM magic, easy to unit test against a temp/in-memory db).
- **HTTP for AI providers**: Dart's plain `http` package, not vendor SDKs — same reasoning as the current app's `ai/` layer (one dependency, easy to mock exact requests/responses in tests, full control over `base_url` overrides for OpenAI-compatible endpoints).
- **State management**: recommend **Riverpod** — plays well with dependency injection (mirrors the current app's constructor-injected controllers), testable without a widget tree, large ecosystem. (Open to Provider/Bloc if preferred — flagging as a decision point, not a foregone conclusion.)
- **File picking / bulk import**: `file_picker` .
- **Local notifications**: `flutter_local_notifications` .

- **Cloud storage integrations**: official Dropbox/Google Drive/Microsoft Graph (OneNote) SDKs or their REST APIs directly via `http`, consistent with the raw-HTTP preference above.
- **Text extraction**: `.txt` / `.md` trivial; `.pdf` / `.docx` / `.pptx` / `.xlsx` need Dart packages per format (needs a short survey — Dart's ecosystem here is thinner than Python's `pypdf` / `python-docx` / `python-pptx`; this is a concrete early risk to spike on, not assume).

---

8. Testing strategy

Good news: Flutter's widget-testing story is materially better than Tkinter's for this purpose — `flutter_test` can drive real widget trees headlessly without needing a display/Xvfb workaround at all, so the smoke-test gap the current app had to explicitly accept (`Docs/testing.md` §"Excluded") likely doesn't need the same trade-off here. Plan:

- Pure logic (scheduling, keyword extraction/linking, grading) — plain Dart unit tests, no widgets involved, same philosophy as `core/` today.
- Repositories — tests against a real temporary/in-memory sqlite db, same philosophy as `storage/` today.
- AI providers — HTTP mocked (e.g. `http`'s `MockClient` or `mocktail`), same philosophy as `ai/` today: every error branch covered without real network calls.
- Widgets/screens — real Flutter widget tests (`testWidgets`), which can actually assert on rendered output headlessly; a genuine upgrade over what was possible with Tkinter.

---

9. Phased build order

1. ~~**Spike / risk reduction**~~ — done, see §11.
2. ~~**MVP core loop**~~ — done, see §11.
3. **Automatic generation**: on-open due-check + generation, local notifications reminding the user to open the app. **Not built** — see §11 (blocked on real-device verification).
4. ~~**Quiz editing + traceability UI**~~ — done, see §11.
5. ~~**Bulk import + staging/drag-and-drop preview screen**~~ — done (with one honest caveat), see §11.
6. **Cloud storage integrations**: Dropbox, Google Drive, OneNote. **Not built** — see §11 (blocked on OAuth app credentials only you can create).
7. ~~**Full data export.**~~ — done, see §11.
8. **Design pass**: match savepoint.cc precisely. **Not done** — still blocked on real brand assets (see §2, §11).
9. **Packaging & store submission**: iOS (Mac + Xcode + Apple Developer account) and Android (Play Console). **Not attempted** — needs your own developer accounts; see §11.

---

10. Open questions / risks to resolve before or during build

- Do quizzes get independent folder placement, or always inherit their source note's folder? **Resolved as built**: a question always belongs to its source note's folder (via `note_id`); no independent quiz-

folder concept was added, since it would've been a second organizing chore for no clear win yet.
- Which cloud integration matters most, to prioritize? Still open — moot until OAuth credentials exist at all (§11).
- Dart text-extraction library availability — **resolved**, see §11.
- Exact savepoint.cc brand assets — still open, see §11.
- Local notification permission/reliability on iOS vs. Android — still open, see §11.
- No migration path from the existing desktop app's data — still the decision as of v1; unchanged.

---

11. Status as of the last work session

Built, working, and covered by automated tests (45 tests passing, `flutter analyze` clean):

- Local-only auth gate (hardcoded admin/1234 behind a swappable `AuthService`), four-tab shell (Notes/Quiz/Account/Settings).
- Notes: folder tree (create/browse nested folders), note CRUD, single-file import, **bulk import with a real staging/preview/reorder/confirm screen**.
- Text extraction for `.txt` / `.md` / `.pdf` / `.docx` / `.pptx` — `.docx` / `.pptx` hand-rolled with `archive` + `xml` (unzip + parse, same approach `python-docx` / `python-pptx` use internally) since no solid pure-Dart package exists for either; `.pdf` via `syncfusion_flutter_pdf` (free for individuals/small teams under Syncfusion's community license — not fully OSS, worth knowing). `.xlsx` **not implemented** — no note content is naturally spreadsheet-shaped, so this was deprioritized rather than forced.
- AI generation: full `ai/` layer ported (OpenAI + Anthropic providers, injectable `http.Client` for testing, one shared `AIProviderError`), wired to a real "Generate" button in the note editor.
- Quiz: real review loop (grade, reschedule via the ported scheduling algorithm, append to `question_history`), manual question editing, and note → question traceability (both directions: "generated from" on a question, "questions from this note" on a note).
- Settings: default question format, **full data export** (every folder/ note/question/review-history row as one JSON file via the OS share sheet).
- Account: provider/model/base URL/API key together, matching §3's IA.

Known, deliberate gap: bulk import stages a **flat, reorderable list** into one destination folder, not a reconstructed source folder tree — see the caveat in `bulk_import_staging_screen.dart`'s own doc comment. Standard mobile file pickers return individually-picked files, not an enumerated folder tree; there's no reliable cross-platform "pick a whole folder recursively" API the way a desktop file dialog has. Revisit if a real need for nested-source import shows up.

Blocked on things only you can provide (not attempted, not faked):

- **Local notifications** (phase 3) — needs a real device to verify permission prompts and delivery actually work; this session's headless emulator already burned significant time on unrelated input-handling quirks (see the FAB debugging story in conversation history) and isn't trustworthy for verifying platform-permission behavior too.
- **Cloud storage integrations** (phase 6) — Dropbox/Google Drive/OneNote each need a real developer-console app registration (client ID/secret) that only you can create. Building OAuth flow scaffolding

against nonexistent credentials would be unverifiable busywork.

- **savepoint.cc exact brand assets** (phase 8) — a text-only page fetch can't extract real hex codes/fonts/spacing. Current theme is a direct port of `gui/theme.py`'s existing (also somewhat unverified) palette claims. Needs either the real CSS/brand guide or screenshots to check against.
- **Packaging & store submission** (phase 9) — needs an Apple Developer account (+ a Mac), a Play Console account, and is inherently a human/billing action, not something to attempt autonomously.